Agentic State Management

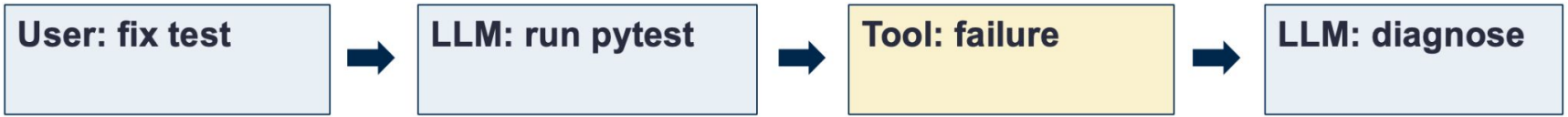
Mohammed Almakrami, Ayan Nair, Vaelone Elankumaran, Himanish Kolli

Continuum: Efficient and Robust Multi-turn LLM Agent Scheduling with KV Cache Time To-Live

Hanchen Li, Qiuyang Mang, Runyuan He, Qizheng Zhang,
Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Alvin Cheung,
Joseph Gonzalez, Ion Stoica

Background and Motivation

An Agent Conversation Creates Reusable Inference State

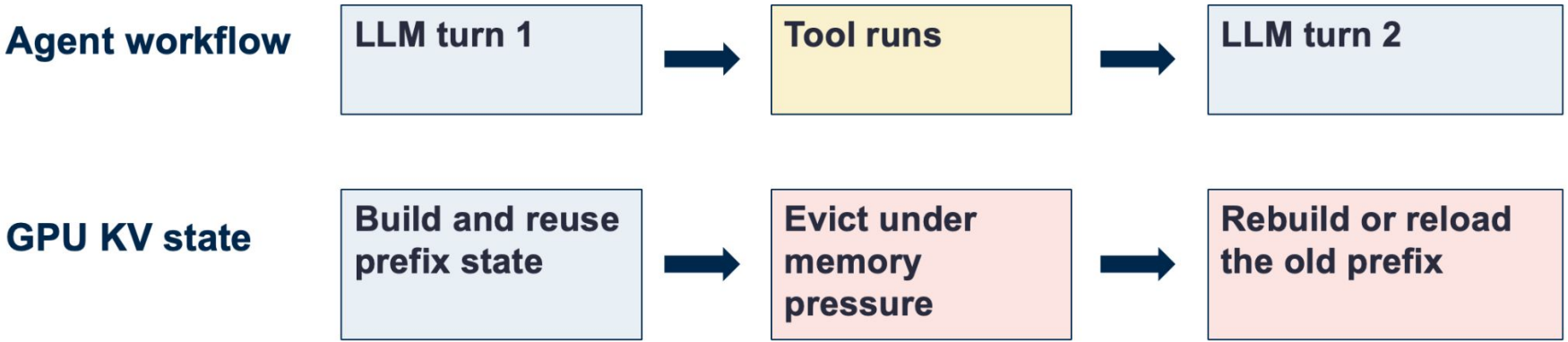


Conversation history stays in the application
The next prompt includes the old messages plus the tool result.

KV cache stores the computation for that shared prefix
Prefill builds it. Decode and later turns can reuse it.

The next LLM turn continues the same program after the tool returns

Default Eviction Can Discard Useful Computation



The full conversation still exists in application memory.

A finished LLM turn can belong to an unfinished agent program

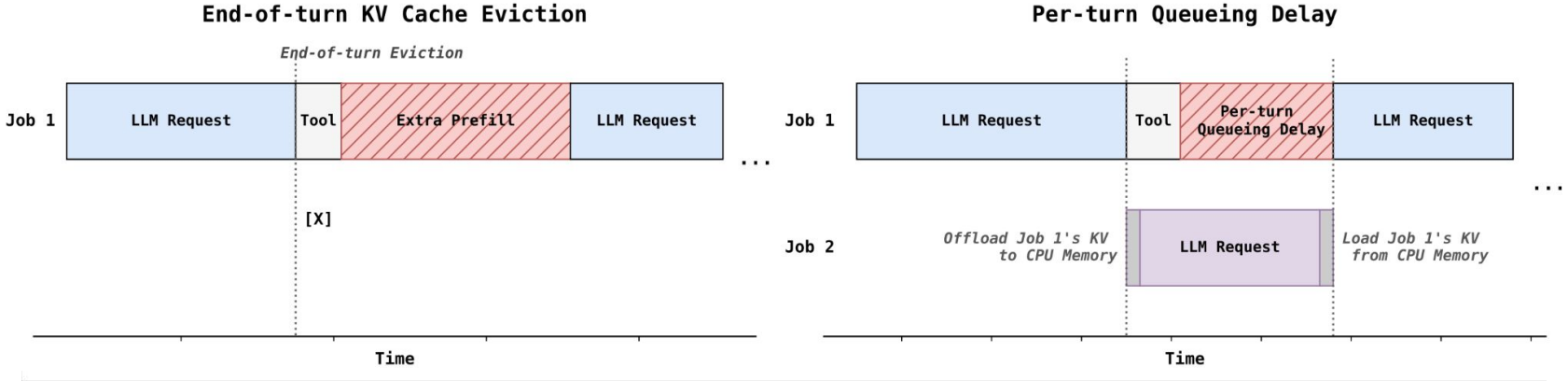
Agent Workloads Contain Many Interruptions

Collected workload	Mean turns	Mean tokens / program
SWE-Bench coding	10.9	≈ 70k
BFCL V4 web search	6.3	≈ 93k

One job revisits inference repeatedly
Each tool pause creates another retention or eviction decision.

Small penalties at each transition accumulate across the whole job

Eviction Adds Two Separate Costs



1. Reconstruction or reload prefix

2. Wait again for GPU admission

Offloading saves cached data, but a returning turn can still queue

Memory Pressure Makes Retention a Tradeoff

Retain during the tool call

Benefit
A quick continuation avoids reconstruction and another wait.

Cost under contention
Pinned memory displaces work from other active jobs.

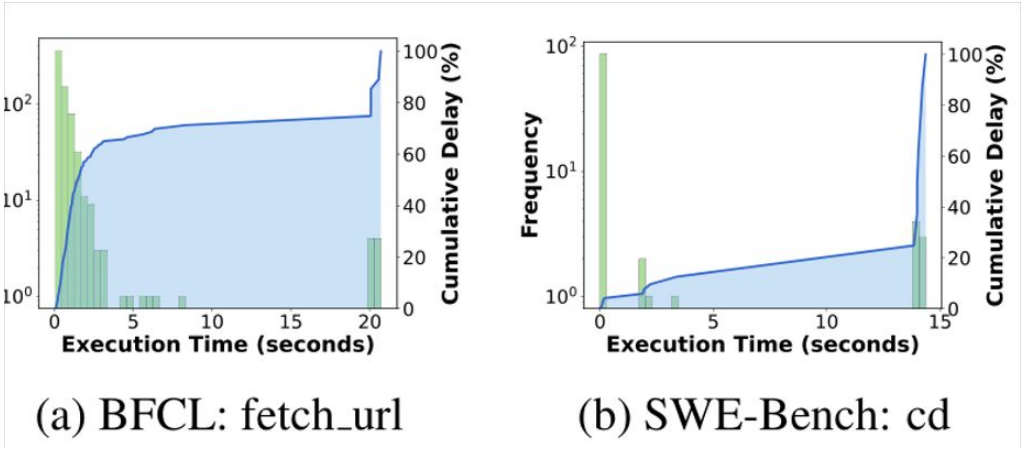
Evict when the turn ends

Benefit
Other jobs can use the released GPU memory immediately.

Cost on return
The agent must reconstruct or reload its prefix and may queue.

The question is how long to retain state when memory is scarce

Why Always Retaining the Cache Fails



(a) BFCL: fetch_url (b) SWE-Bench: cd

Tool types differ

One fixed retention interval misses these differences.

Individual calls vary
A short expected call can still take much longer.

Estimate each tool's return-time distribution, then bound retention

How Long should Paused KV State Stay on the GPU?

Save the cost of resuming
Avoid rebuilding the prefix and repeatedly re-entering the queue.

Account for competing programs
Pinned state occupies memory that other jobs could use.

Handle uncertain return times
A retention decision should adapt to the tool and expire when needed.

Optimize the full agent job across LLM and tool interruptions

Methodology

Continuum Retains State for a Bounded Interval



The tool returns within the retention window
Reuse pinned state and prioritize the program's continuation.

The tool takes longer than expected
Expiry releases idle pins. Forced unpinning resolves allocation deadlock.

Cache retention and program-level priority work together

Where the Policy Runs

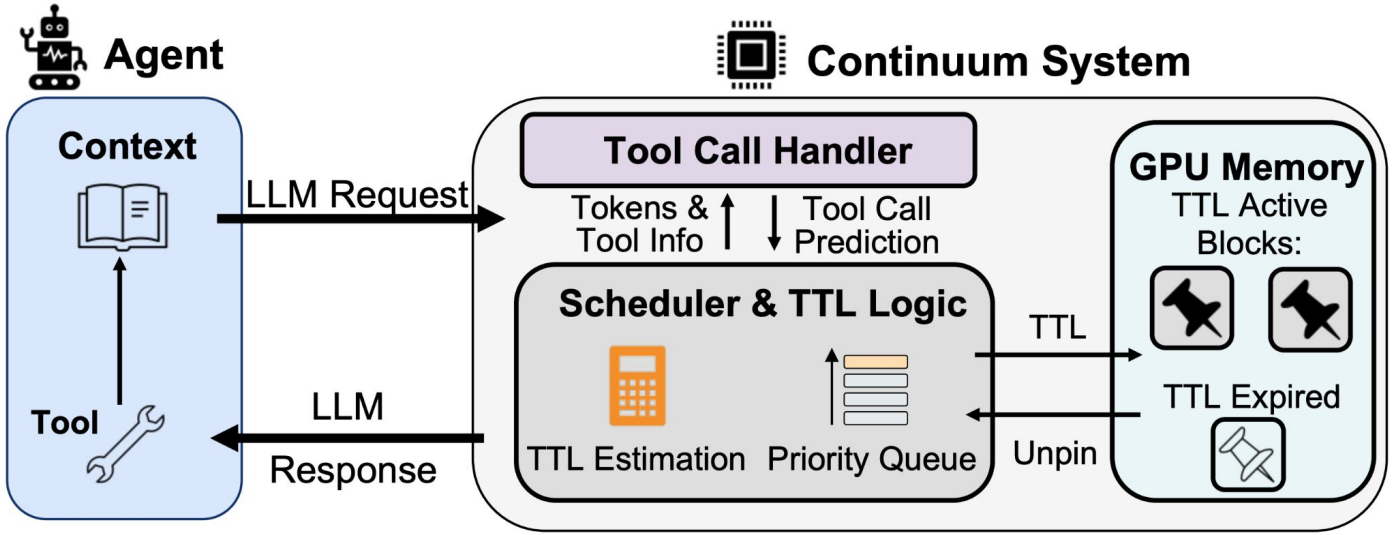


Figure 13: *System Overview of Continuum*

A tool-call handler connects program history to KV retention and scheduling

Choosing TTL

For request r and its tool f:

τ: candidate retention time
$P(\tau, f)$: chance the tool returns by τ

$B(r)$: latency saved if state is reused
$C(\tau, r)$: cost of holding GPU memory

$$\tau^* = \text{argmax}_\tau \left[P(\tau, f) \times B(r) - C(\tau, r) \right]$$

A longer TTL raises reuse probability, but also holds memory longer

Example of the TTL Decision

Observed tool times: 0.1, 0.2, 0.2, 3.0 s

Assume a reuse saves 4 s and retention costs τ s.

TTL τ	P(return by τ)	Expected benefit	Net benefit
0 s	0%	0 s	0 s
0.1 s	25%	1.0 s	0.9 s
0.2 s	75%	3.0 s	2.8 s
3.0 s	100%	4.0 s	1.0 s

0.2 s wins: waiting for the last slow call costs more than it saves

Evaluation

Evaluation Setup

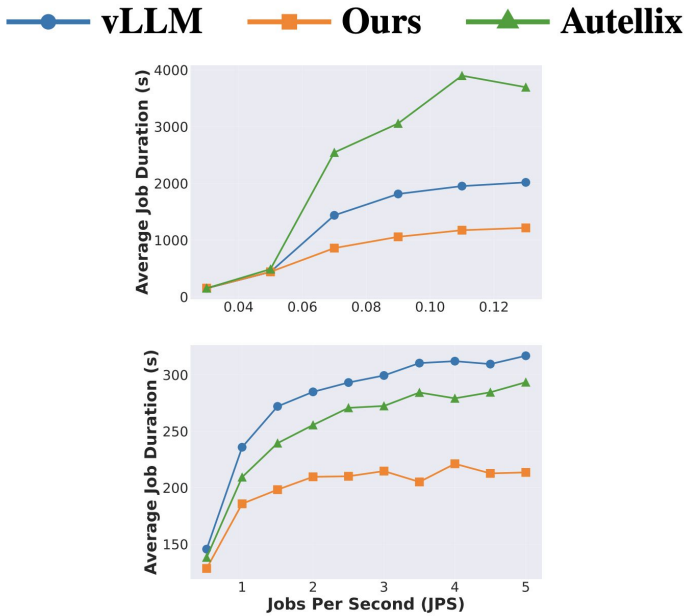
Workloads and serving model
SWE-Bench coding and BFCL V4 web search. Llama 3.1 8B / 70B.

Selected configuration
Llama 3.1 8B on one B200. Poisson job arrivals.

Metric and baselines
Whole-job duration. Compare vLLM, Autellix, and component ablations.

Most experiments replay collected traces, isolating serving behavior

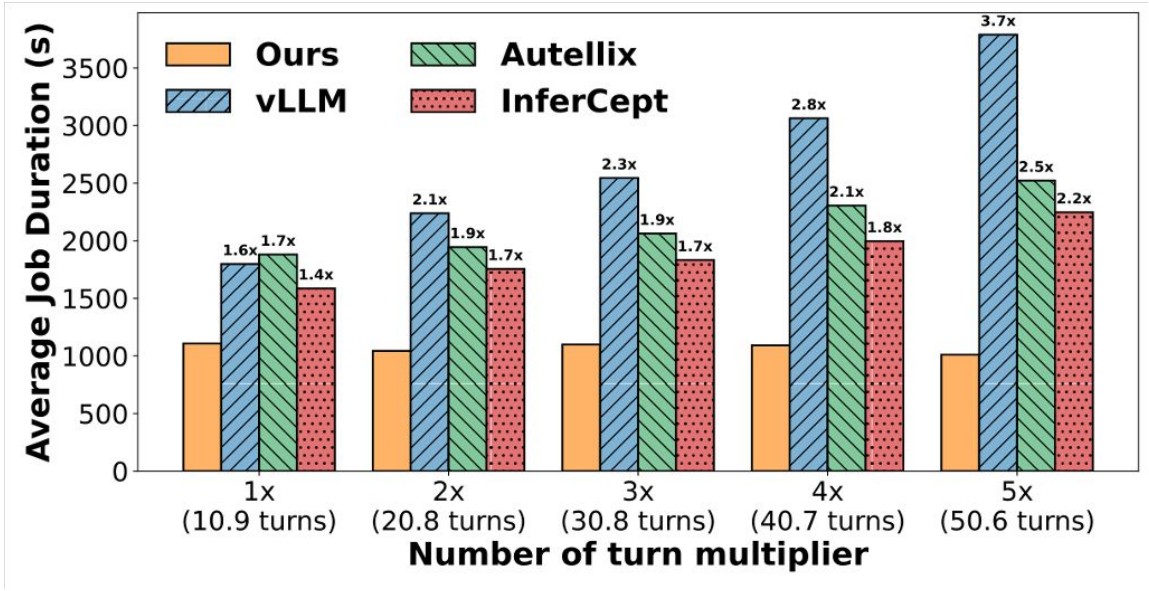
Lower Job Duration across Workloads



Llama 8B (1×B200)

Continuum outperforms against baseline schedulers

More Turns Amplify the Cost of Interruptions



Controlled test
Repeat traces 1× to 5× and shorten tokens per turn.

What it supports
Repeated interruptions make continuity increasingly valuable.

Strengths and Limitations

Strengths

Models reconstruction and queueing together

Bounds cache retention during slow tools

Works alongside CPU offloading

Limitations

Depends on historical latency estimates

Cost model approximates memory contention

Sequential agent flow limits generality

A useful serving policy with workload-dependent assumptions

Future Research Directions

Branching agents: account for shared prefixes and concurrent continuations.

Changing tools: adapt latency histories when execution conditions shift.

Fairness: bound waiting time for newly arrived and short programs.

Broader evaluation: test live workloads with heterogeneous agent behavior.

State lifetime policies should evolve with agent control flow

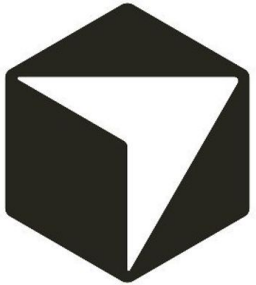
STRATA: HIERARCHICAL CONTEXT CACHING FOR LONG-CONTEXT LLM SERVING

Zhiqiang Xie, Ziyi Xu, Mark Zhao, Yuwei An, Vikram Sharma Mailthody, Scott Mahlke, Michael Garland, Christos Kozyrakis

Background and Motivation

Industry Context

- LLMs have become very prominent
- Many use cases now rely on long-context windows
  - Coding Assistants (Claude Code, Codex, etc.)
  - Retrieval Augmented Generation
  - Document Analysis
  - Conversational LLMs with long-term memory
- Consequently, more KV cache data must be managed

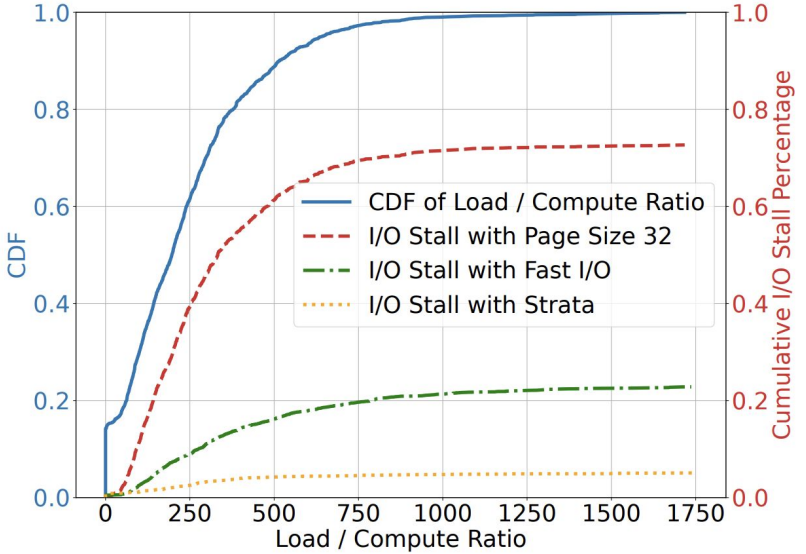






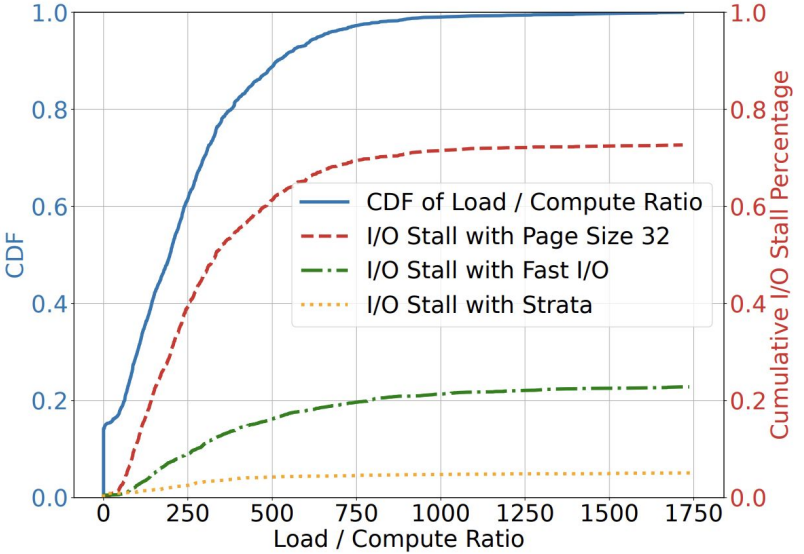
Consequences of Increased KV Cache Data

- More Total Entries - > More cache misses
- GPU memory, even GPU + CPU memory is not sufficient to hold all of this
  - Hierarchical Memory - data transfers required
  - GPU, CPU, SSD, Memory Pools
  - Inflates TTFT(Time To First Token)
  - Degrades Throughput by 4X
  - 74% of Prefill is spent on I/O Stall



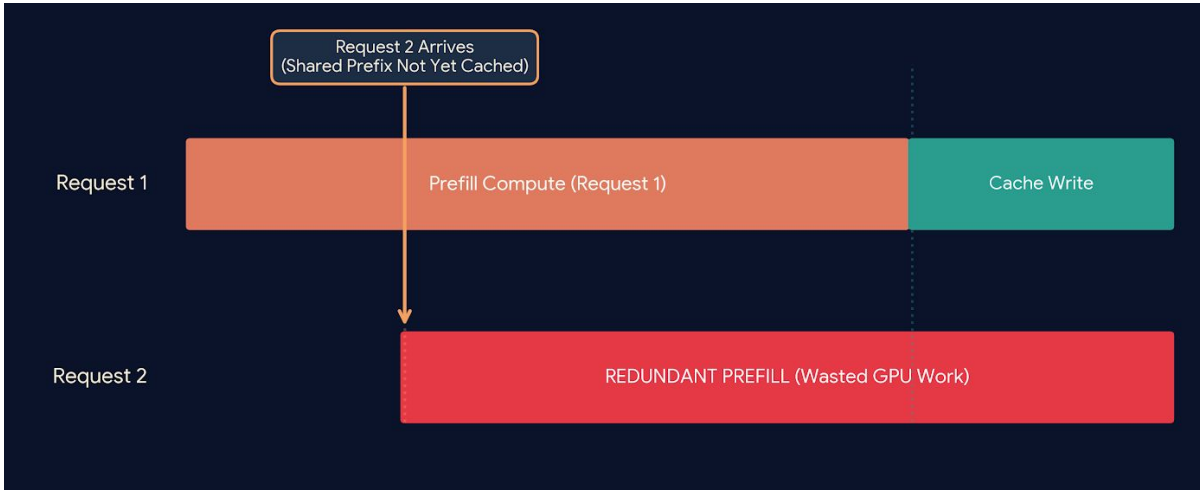
Scheduling Flaws - Data Transfers

- In small context scenarios, computing new tokens dominates data transfers
- Thus, even in long context scenarios, transfers assumed as negligible by scheduler
- This assumption does not necessarily hold



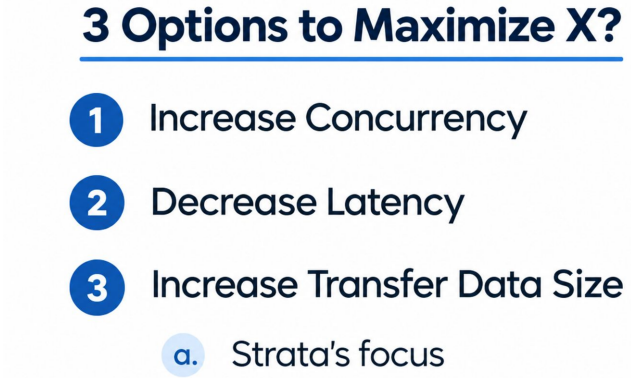
Scheduling Flaws - Delay Hit Phenomenon

- Multiple requests sharing a context prefix arrive close together, but the shared prefix is not yet cached.
- Before first request caches the prefix, the scheduler begins prefill for subsequent requests.
- This causes redundant prefill computation, wasting GPU resources and increasing latency.



Increasing Throughput

- Consider the following parameters
  - λ = arrival rate of I/O operations
  - C = average number of concurrent I/O operations
  - L = average latency per operation
  - S = average data size per I/O operation
- Formulating Throughput (X) via Little's Law
  - Concurrency: $C = \lambda \times L$
  - Throughput: $X = \lambda \times S$
  - Combining yields: $X = (C \times S) / L$



Page-Size Tradeoff

- To increase transfer size expected first step would be page size, but there are tradeoffs

	Smaller Pages	Larger Pages
Memory Utilization	Better	Worse
Transfer Speed	Slower	Faster
Cache Hit Rate	Higher	Lower

- We need to find an approach which can take the best of both

Summary: What Are We Looking for in a Solution?

- Optimize KV Cache Transfers
  - Increase GPU–CPU transfer efficiency by transferring larger amounts of data per operation.
- Preserve Memory Efficiency
  - Avoid wasting memory on oversized pages and maintain a high cache hit rate.
- Co-optimize Across the Memory Hierarchy
  - Treat GPU–CPU bandwidth as a first-class resource alongside GPU compute and HBM capacity.

Methodology

Strata Architecture

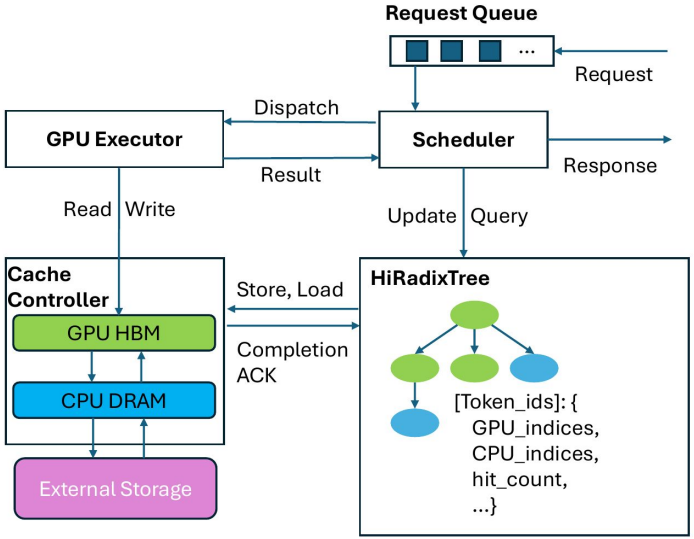


Figure 4: System Architecture of Strata

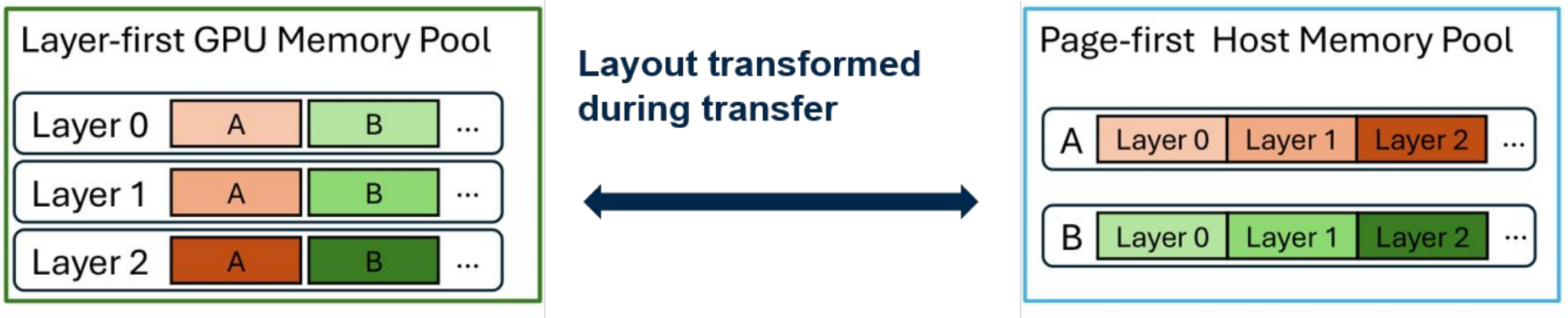
Scheduler
Selects requests and forms batches.

HiRadix Tree
Tracks KV-cache metadata and locations

Cache Controller
Loads, backs up, and evicts KV caches

Efficient KV-Cache I/O

Small pages, efficient transfers



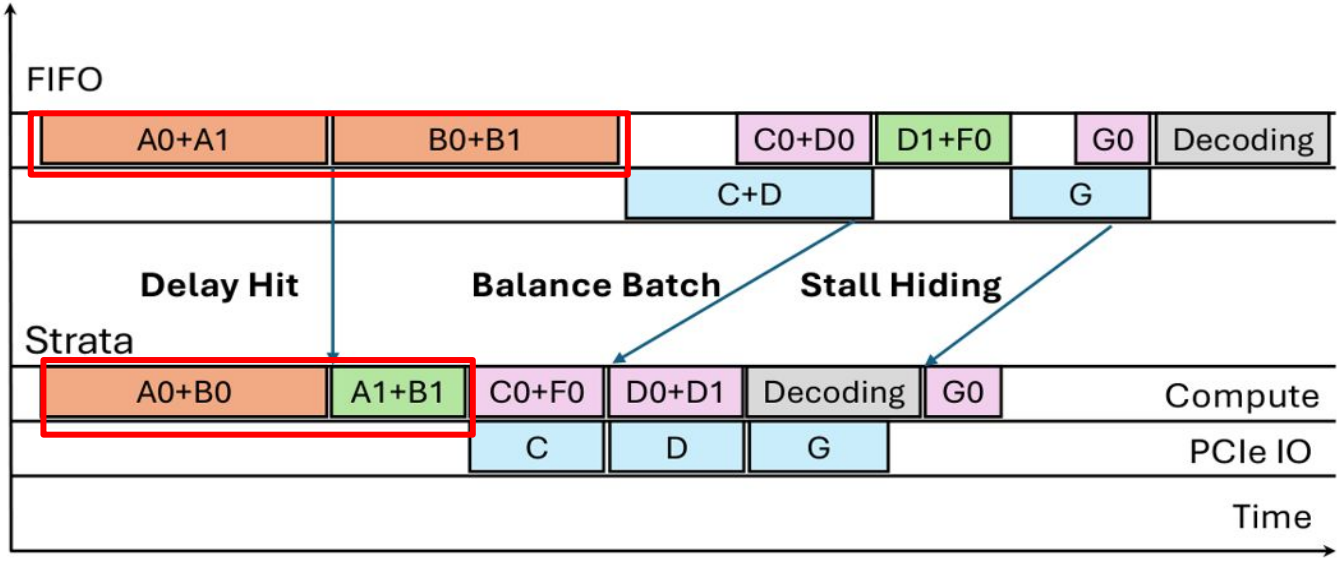
GPU: Layer-first
Organizes KV data for layer-wise computation

Host: page-first
Keep each page's layers together

GPU-Assisted I/O Threads copy fragments concurrently and transform the layout during transfer

Cache-Aware Scheduling

X□ is the nth request using context X



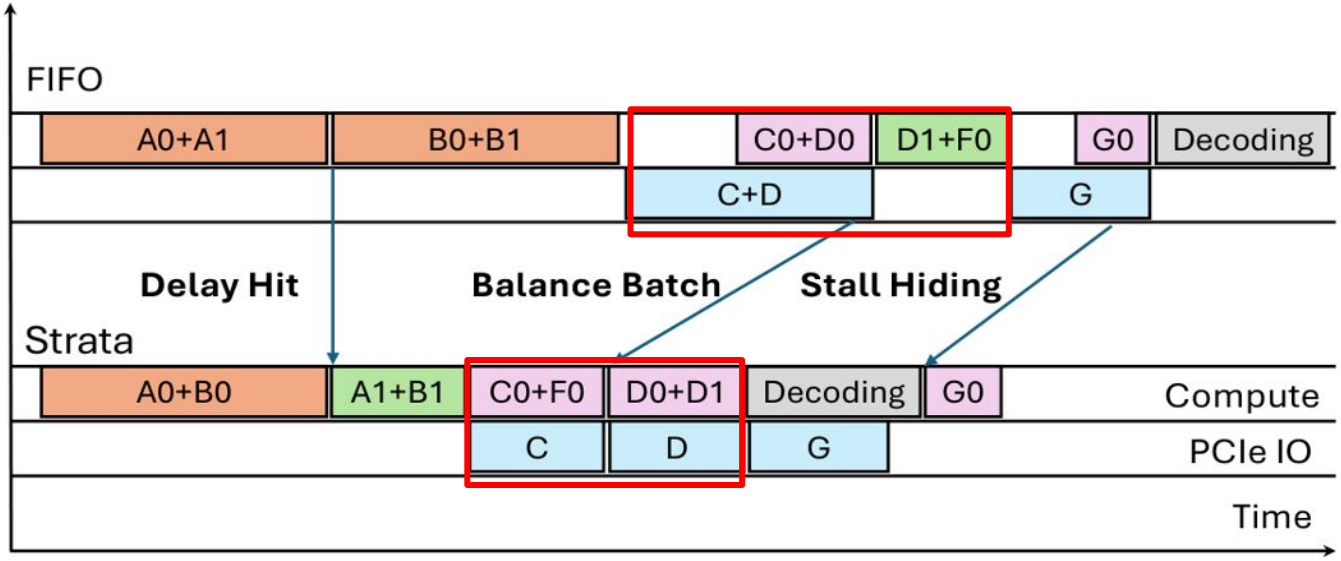
Orange: cache miss · Green: GPU hit · Purple: host hit · Blue: transfer · Gray: decode

1. Delay-Hit Deferral

Defer requests that can reuse a context being computed

Cache-Aware Scheduling

X□ is the nth request using context X



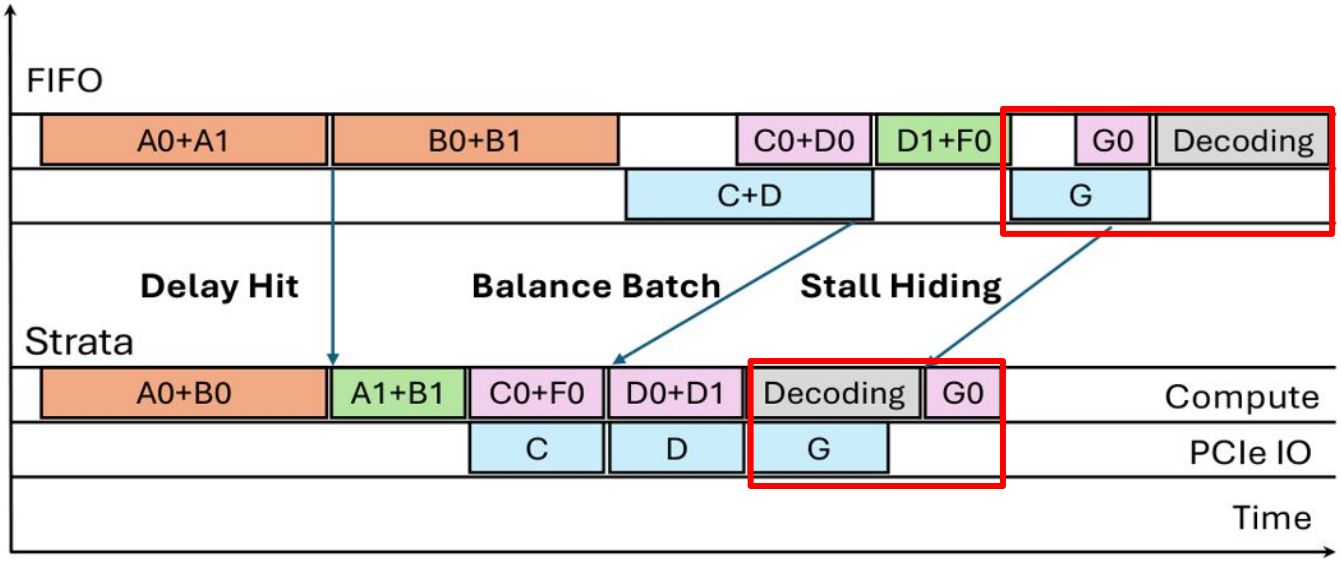
Orange: cache miss · Green: GPU hit · Purple: host hit · Blue: transfer · Gray: decode

2. Balanced baching

Pair cache loading with enough computation to overlap it

Cache-Aware Scheduling

X□ is the nth request using context X



Orange: cache miss · Green: GPU hit · Purple: host hit · Blue: transfer · Gray: decode

3. Bubble filling (stall hiding)

Run useful computation during remaining loading stalls

Evaluation

Evaluation Setup

Main platform: NVIDIA H200

Models

Llama-3.1-8B-Instruct
Qwen2.5-14B-Instruct-1M
Llama-3.1-70B-Instruct

Main hierarchical-cache baselines

SGLang-HiCache
vLLM-LMCache
TRT-LLM-HiCache

Workloads

LooGLE — long-document QA
NarrativeQA — very long-context reading comprehension
ReviewMT — multi-agent conversations
ShareGPT — short conversations

Metrics

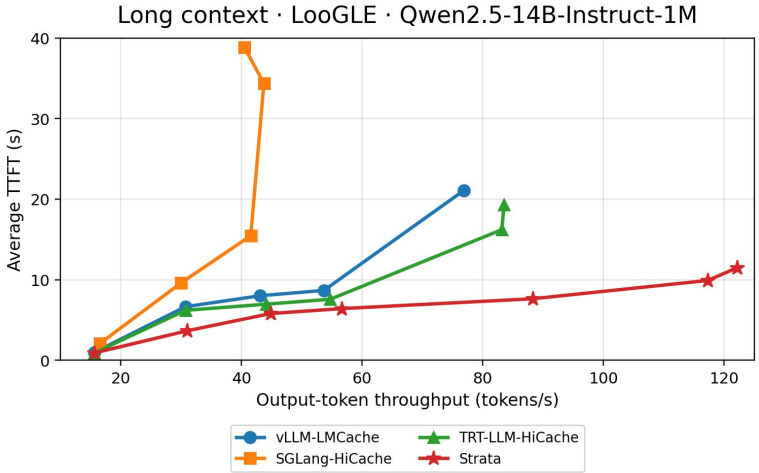
Average TTFT
Output-token throughput

End-to-End Performance

Qwen2.5-14B-Instruct-1M

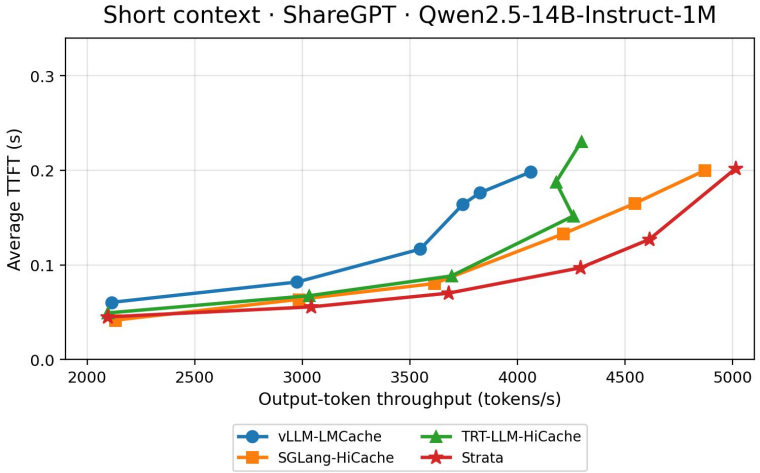
Long context · LooGLE

Higher throughput at comparable TTFT



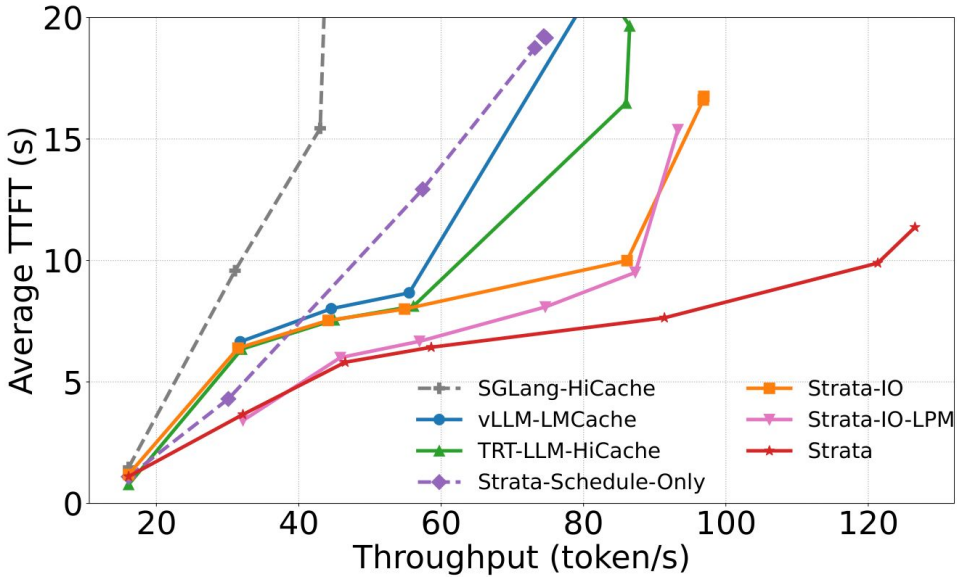
Short context · ShareGPT

Performance remains comparable



Why Does Strata Work?

Both I/O optimization and scheduling contribute to the improvement



Strata-IO

GPU-assisted I/O only

Strata-Schedule-Only

Cache-aware scheduling only

Strata

Both mechanisms together

Lower offered request rates

Scheduling can hide much of the lighter I/O pressure

Higher offered request rates

I/O becomes the bottleneck, efficient transfers matter more

Discussions

Discussion & Limitations

What does Strata still not solve?

GPU interference
GPU-assisted I/O still consumes some SM resources

Future direction
Reduce I/O-kernel overhead

Fairness & SLOs
Aggregate efficiency can still treat requests unevenly

Future direction
Fairness- and SLO-aware scheduling

Current scope
Focuses on dense KV caches and host DRAM → GPU HBM loading

Future direction
Hybrid models and slower storage tiers

Strata improves system efficiency, but per-request guarantees and broader model/storage support remain open.

Thank you